

Bilgisayar Mühendisliği Bölümü
BIMU2057 Veri Yapıları

Dr. Öğr. Üyesi Nihal ALTUNTAŞ

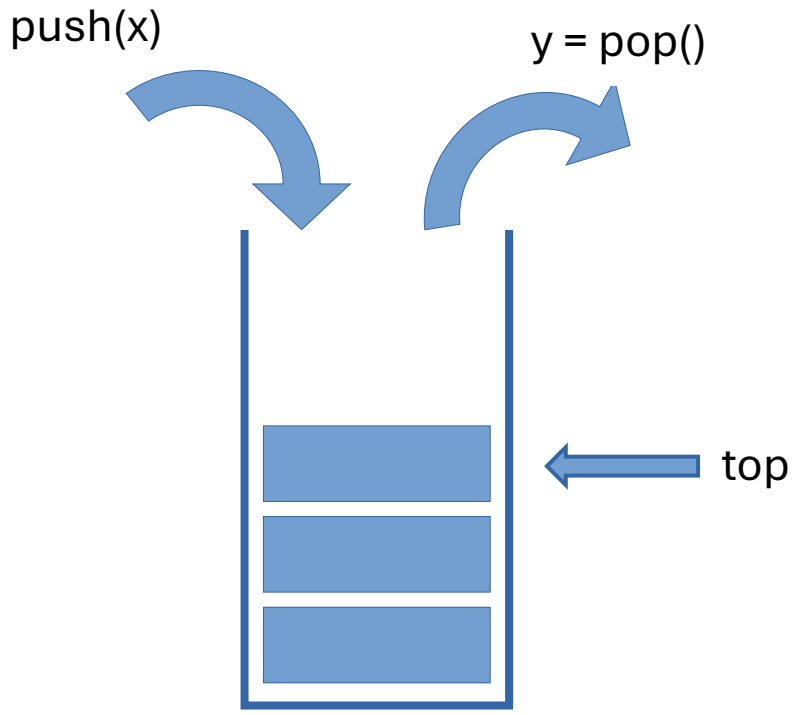
nihal.altuntas@iuc.edu.tr

Bu Derste

- Stack (Yığın)
  - Stack Uygulama Alanları
  - Genel Stack İşlemleri
  - Sabit kapasiteli dizi ile stack
  - Bağlı liste ile stack
  - Örnek Uygulama: parantez kontrolü
- Queue (Kuyruk)
  - Queue Uygulama Alanları
  - Genel Queue İşlemleri
  - Dizi ile Queue
  - Linked List ile Queue

Stack (Yığın)

- Yığın, elemanların yalnızca bir uçtan (tepe/top) eklenip çıkarıldığı doğrusal bir veri yapısıdır.
  - Bu özellik LIFO (Last In First Out) olarak bilinir.



Stack Uygulama Alanları

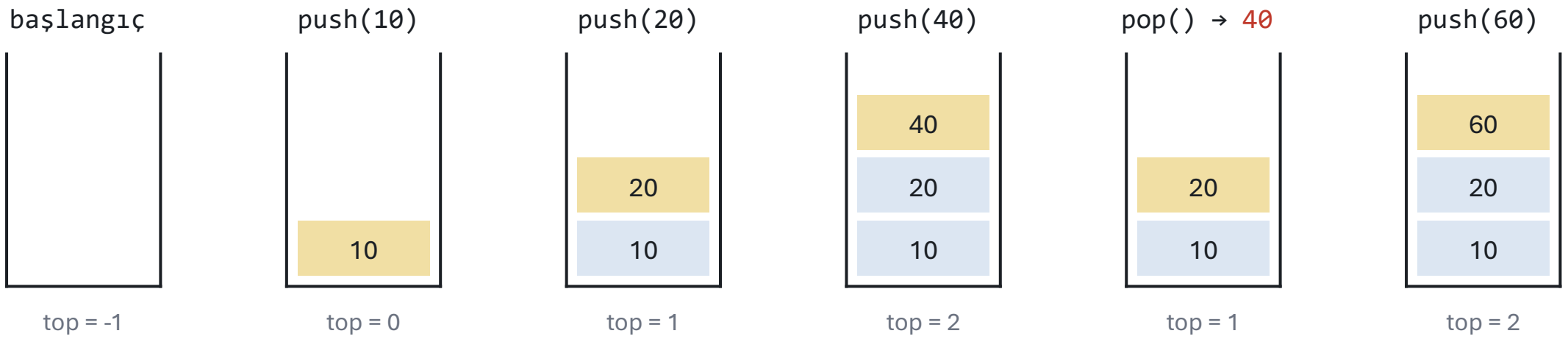
- Stacklar programlamada geriye takibe (backtracking) ihtiyaç olduğu durumlarda kullanılır
  - Fonksiyonların çağırılması
    - Recursion
  - Yazım / terim kontrolü
  - Geri al / Yinele işlemleri

Genel Stack İşlemleri

İşlem	Türkçe	Açıklama	Karmaşıklık
push(node)	ekle(node)	Tepeye yeni eleman ekler	O(1)
pop()	cikar()	Tepedeki elemanı çıkarır ve döndürür	O(1)
peek()	tepe()	Tepedeki elemanı çıkarmadan döndürür	O(1)
isEmpty()	bosMu()	Stack boş mu?	O(1)
isFull()	doluMu()	Stack dolu mu? (yalnızca dizi ile)	O(1)

Tüm işlemler tek uçta (tepe) yapıldığı için eleman sayısından bağımsız, sabit zamanda çalışır.

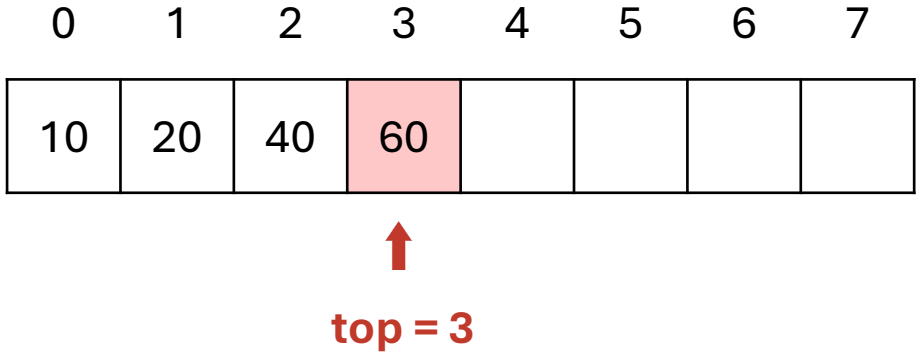
Adım Adım Stack



Sarı kutu her adımda tepedeki elemandır. **pop()** her zaman en son eklenen elemanı döndürür: çıkış sırası, giriş sırasının tersidir.

Sabit Boyutlu Array ile Stack Kullanımı

- Sabit boyutlu bir array ve tepedeki elemanın indisi `top` ile tutulur.
- Ekleme öncesi dolu mu? Kontrolü
  - Taşma (overflow) önlenir.
- Çıkarma öncesi boş mu? Kontrolü
  - Alttaşım (underflow) önlenir.
- Stack
  - Boşken `top = -1`
  - Doluyken `top = kapasite - 1`



Sabit Boyutlu Array ile Stack Kullanımı

```
class ArrayStack:
    def __init__(self, capacity):
        self.capacity = capacity
        self.arr = [None] * capacity
        self.top = -1

    def isEmpty(self):
    def isFull(self):
    def push(self, node):
    def pop(self):
    def peek(self):
```

Linked List ile Stack Kullanımı

- Dizi kullanarak gerçeklemenin getirdiği kısıtlamalar, linked list kullanılarak kaldırılabilir.
- Linked List işlemlerinde listenin başına eklemek ve çıkartmak daha hızlıdır.
  - Listenin başına ekleme ve baştan silme O(1)

Linked List ile Stack Kullanımı

- Bu nedenle, stack'in tepe noktası linked list'in ilk elemanı (head/top) olacak şekilde tasarlanır.
  - Ekleme işlemi – Her zaman, listenin başına bir eleman ekleme
  - Çıkarma işlemi – Her zaman, listenin başından bir eleman çıkarma

`push(node)` = Linked list'te **insertFirst** (başa ekle)

`pop()` = Linked list'te **removeFirst** (baştan sil)

Linked List ile Stack Yapısı

```
class LinkedStack:
   def __init__(self):
      self.top = None

   def printStack(self):

   def isEmpty(self):
   def push(self, eleman):
   def pop(self):
   def peek(self):
```

```
class Node:
   def __init__(self, newData):
      self.data = newData
      self.next = None
```

Array mi Linked List mi?

Özellik	Array ile stack	Linked list ile stack
Kapasite	Sabit, baştan belirlenir	Dinamik, ihtiyaca göre
Taşma (overflow)	Olabilir	Olmaz (bellek bitmedikçe)
push / pop süresi	O(1)	O(1)
Ek bellek	Kullanılmayan hücreler boşa gider	Her düğümde bir next referansı
Bellek yerleşimi	Ardışık, önbellek dostu	Dağınık

Örnek Uygulama

- Parantez Dengesi Kontrolü

İfade: { [( )] } — her açılan parantez doğru sırayla kapanıyor mu?

Okunan	İşlem	Yığın (tepe sağda)
{	Açılış → push	{
[	Açılış → push	{ [
(	Açılış → push	{ [(
)	Tepe (ile eşleşir → pop	{ [
]	Tepe [ile eşleşir → pop	{
}	Tepe { ile eşleşir → pop	(boş)

Kurallar

Açılış → push

Kapanış → tepeyle eşleşmeli, pop

Sonda yığın boş olmalı

Sonuç: Dengeli ✓

Parantez Kontrolü Pseudocode

```
dengeliMi(ifade)
   yigin = Stack()
   for k in ifade
      if k in "([{"
         yigin.push(k)
      else if k in ")]}"
         if yigin.isEmpty()
            return False
         acik = yigin.pop()
         if not eslesir(acik, k)
            return False
   return yigin.isEmpty()
```

Dengesiz örnekler

`( [ ) ]`

")" gelince top "[" → eşleşmez

`( ( )`

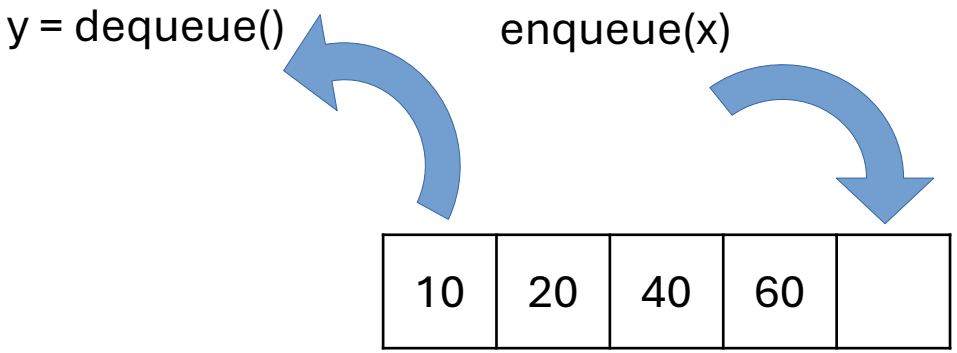
Sonda yığın boş değil

`( ) )`

")" gelince yığın boş

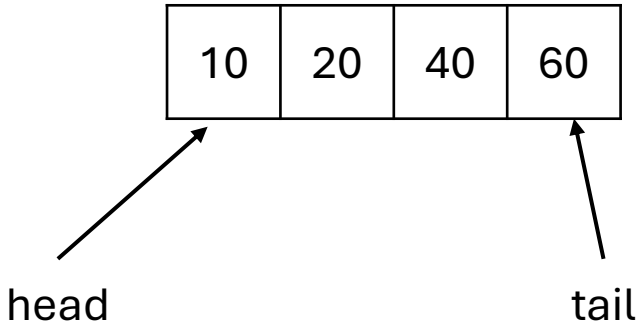
Kuyruk (Queue)

- Temel veri yapılarından bir diğeri kuyruktur.
- Yığının aksine, kuyruğa ilk giren eleman kuyruktan ilk çıkar
  - Bu özellik FIFO (First In First Out) olarak bilinir.



Kuyruk (Queue)

- Kuyruğa ekleme ve kuyruktan çıkarma işlemleri farklı uçlardan yapıldığı için kuyruk yapısının başı ve sonu tutulur.
  - Yeni eklenen veriler kuyruğun sonuna eklenir.
  - Çıkarma işlemi kuyruğun başından yapılır.



Kuyruk Uygulama Alanları

- Kuyruk bir bekleme dizisidir. Belirli bir servis için girdilere bekleme işlemi uygulamak için kullanılır.
  - CPU Scheduling: Bilgisayar işlemcisinin tek çekirdeği bir anda sadece tek bir işlem gerçekleştirebilir. İşlemcinin çalıştıracağı programlar bir kuyrukta tutulur.
  - Farklı hıza sahip sistemlerin iletişiminde arabellek (buffer) olarak kullanılır.
  - Gerçek dünya uygulamaları

Genel Kuyruk İşlemleri

İşlem	Türkçe	Açıklama	Süre
enqueue(eleman)	ekle(eleman)	Kuyruğun sonuna eleman ekler	O(1)
dequeue()	cikar()	Baştaki elemanı çıkarır ve döndürür	O(1)*
peek()	bas()	Baştaki elemanı çıkarmadan döndürür	O(1)
isEmpty()	bosMu()	Kuyruk boş mu?	O(1)
isFull()	doluMu()	Kuyruk dolu mu? (yalnızca dizi ile)	O(1)

* Doğru uygulanırsa. Lineer dizide dequeue tüm elemanları kaydırır ve O(n) olur — birazdan göreceğiz.

Adım Adım Queue

`enqueue(10)`	10			
`enqueue(20)`	10	20		
`enqueue(40)`	10	20	40	
`dequeue() → 10`	20	40		
`enqueue(60)`	20	40	60	
`dequeue() → 20`	40	60		

Mavi hücre her adımda **head**'deki elemandır.

Çıkış sırası = giriş sırası: 10, 20, ...

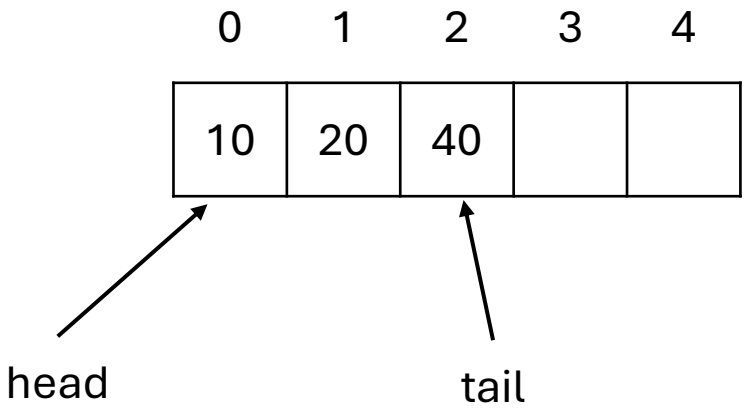
Aynı işlemleri yığına uygulasaydık dequeue yerine pop() 40'ı ve 60'ı döndürürdü.

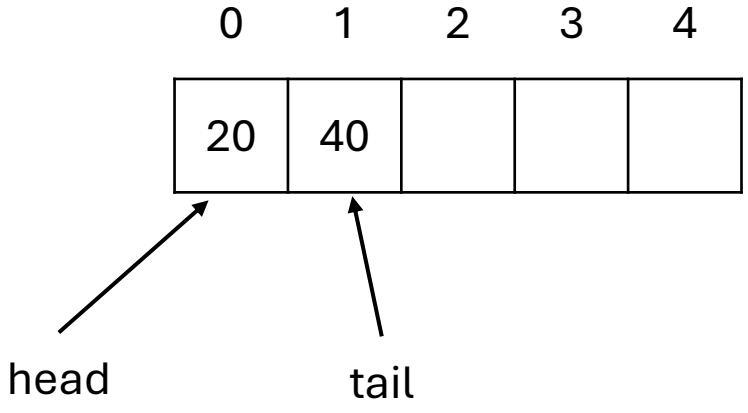
Array ile Kuyruk Kullanımı

- Boyutu sabit olan ve verileri bir array içerisinde tutan yapıya sahip kuyruktur.
- Temel olarak iki faklı uygulaması yaygındır
  - Lineer kuyruk dizisi
  - Döngüsel kuyruk dizisi
    - Array için ayrılmış olan hafızanın verimli kullanılabilmesini sağlar

Lineer Kuyruk Dizisi

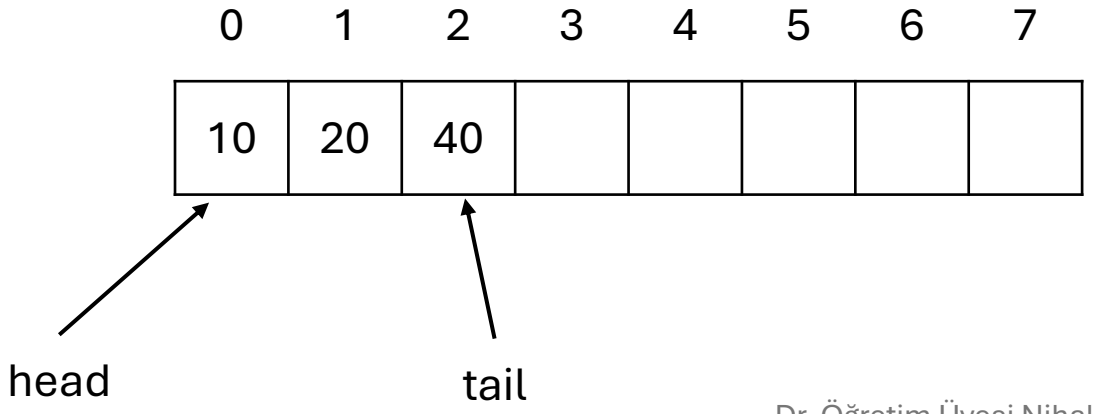
- Kuyruğun başı olarak her zaman dizinin başı kabul edilir.
- Eklemeler sona yapılır.
- Kuyruğun başından çıkarma işlemi sonrası diğer tüm veriler bir kaydırılır.
  - Dezavantaj: Kuyruk boyu uzadıkça işlem maliyeti artar

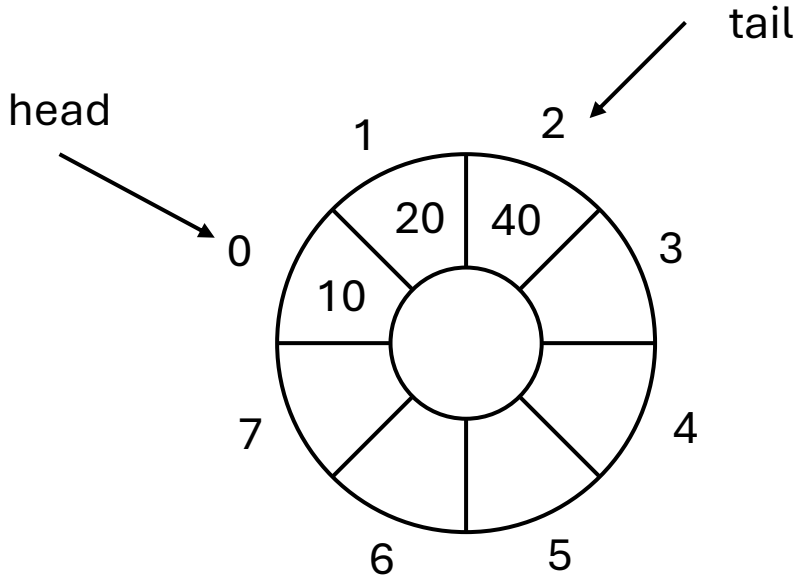




Döngüsel Kuyruk Dizisi

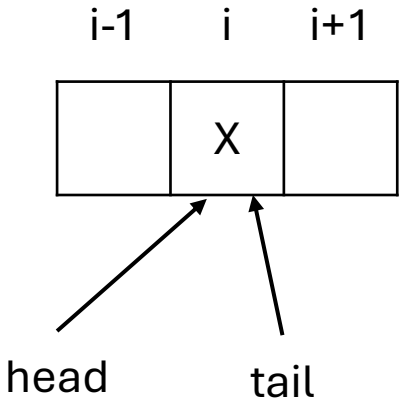
- Dizi doğrusal değil döngüsel olarak kabul edilir.
- Farklı zamanlarda dizinin farklı bölümlerinde elemanlar bulunacaktır.
- `head`: İlk elemanı gösterir.
- `tail`: Son elemanı gösterir



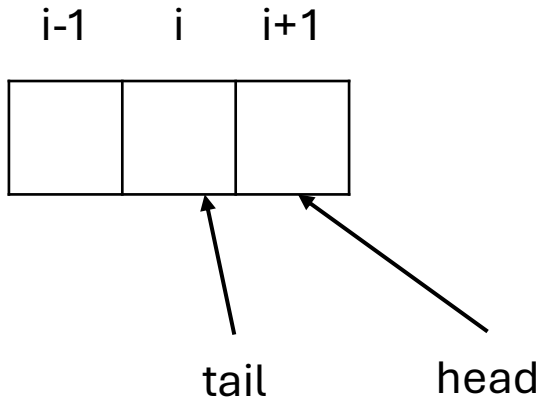


Döngüsel Kuyruk Dizisi Sınır Durumlar

- Kuyrukta tek eleman varsa;



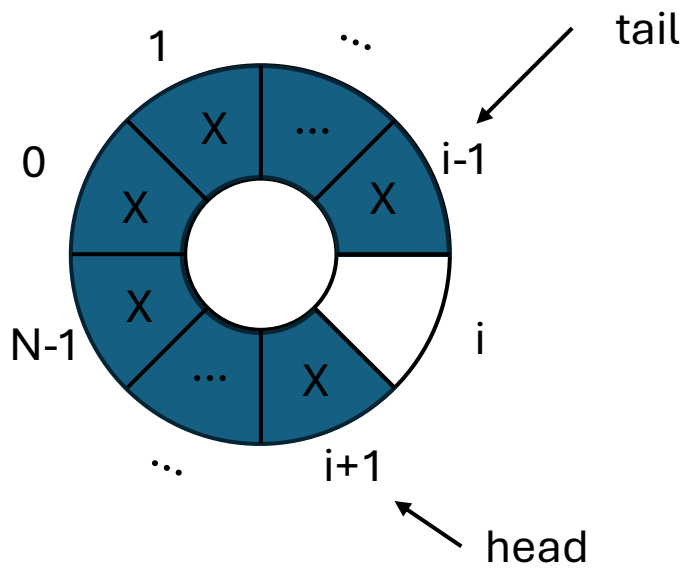
- Kuyruktan son eleman çıkarılırsa;



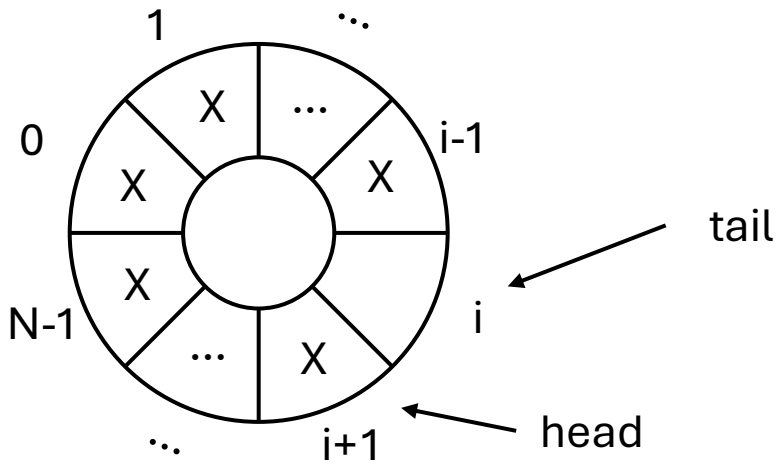
- Kuyruğun boş olma durumu:
  - `head = tail + 1`

Döngüsel Kuyruk Dizisi Sınır Durumlar

- Kuyrukta tek boş yer varsa;



- Kuyruktaki tek boş yere yeni bir eleman eklenirse;



- Kuyruğun dolu olma durumu:
  - `head = tail + 1`

Döngüsel Kuyruk Dizisi Sınır Durumlar

- Kuyruğun boş mu yoksa dolu mu olduğunun kontrolü için başka bir yol bulmak gerekiyor
  - Kuyruktaki eleman sayısını içeren bir değişken
    - Kuyruk boş → `elemanSayisi == 0`
    - Kuyruk dolu → `elemanSayisi == kapasite`
  - Özel indis değerleri: Boş olma durumunu için işaretçilere özel değer ata.
    - İlk durumda kuyruk boş → `head = - 1, tail = -1`
    - Kuyruk tekrar boşalınca ilk durum değerlerini geri ata
    - Kuyruk dolu durumu → `head == (tail + 1) mod kapasite`

Döngüsel Kuyruk Pseudocode (Sayaçlı)

```
__init__(kapasite)
   dizi = [None] * kapasite
   head = 0
   tail = -1
   elemanSayisi = 0

enqueue(eleman)
   if elemanSayisi == kapasite
      print "Kuyruk dolu!"
      return
   tail = (tail + 1) % kapasite
   dizi[tail] = eleman
   elemanSayisi += 1
```

```
dequeue()
   if elemanSayisi == 0
      print "Kuyruk boş!"
      return None
   eleman = dizi[head]
   head = (head + 1) % kapasite
   elemanSayisi -= 1
   return eleman
```

tail = -1 ile başlarsak ilk enqueue'da (−1 + 1) % k = 0 olur.

Linked List ile Queue Kullanımı

- Array kullanarak uygulamanın getirdiği kısıtlamalar, linked list kullanılarak ortadan kaldırılabilir.
- Kuyruğun başı listenin ilk elemanı, kuyruğun sonu ise listenin son elemanı olacak şekilde tasarım yapılır.
  - Kuyruğa ekleme işlemi – listenin sonuna bir eleman ekleme
  - Kuyruktan çıkarma işlemi – listenin başından bir eleman çıkarma
- `head`: Kuyruğun ilk düğümüne işaret eder. Çıkarma işlemi daima `head` ile gösterilen düğümden yapılır.
- `tail`: Kuyruğun son düğümüne işaret eder. Ekleme işlemi daima `tail` ile gösterilen düğümün arkasına yapılır.

Linked List ile Queue Pseudocode

```
enqueue(eleman)
   newNode = Node(eleman)
   if tail == None
      head = newNode
      tail = newNode
   else
      tail.next = newNode
      tail = newNode
```

```
dequeue()
   if head == None
      return None
   eleman = head.data
   head = head.next
   if head == None
      tail = None
   return eleman
```

Boş kuyruğa ekleme: ilk düğüm hem head hem tail olur.

Son eleman çıkınca: tail da None yapılmalı; yoksa silinen düğümü göstermeye devam eder.

Python'da Hazır Yapılar

Yığın → list

```
yigin = []
yigin.append(10)   # push
yigin.append(20)
yigin[-1]          # peek → 20
yigin.pop()        # pop → 20
len(yigin) == 0    # isEmpty
```

Kuyruk → collections.deque

```
from collections import deque
kuyruk = deque()
kuyruk.append(10)   # enqueue
kuyruk.append(20)
kuyruk[0]           # peek → 10
kuyruk.popleft()    # dequeue → 10
```

Kuyruk için list.pop(0) kullanmayın: tüm elemanlar kayar → O(n), lineer kuyruk dizisiyle aynı sorun. deque iki uçta da O(1)'dir.

Özet: Stack ve Kuyruk

Özellik	Stack	Kuyruk
İlke	LIFO	FIFO
Ekleme	push → top	enqueue → tail
Çıkarma	pop ← top	dequeue ← head
Tutulan referans	top	head ve tail
Dizi ile	tepe indisi	döngüsel dizi + mod
Bağlı liste ile	başa ekle, baştan sil	sona ekle, baştan sil
Python	list	collections.deque

Referanslar

- Introduction to Algorithms, Third Edition, Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein, The MIT Press, 2009
- Algorithms, 4th Edition by Robert Sedgewick and Kevin Wayne, Addison-Wesley Professional, 2011
- Data Structures and Algorithms in C++, Third Edition, Adam Drozden, 2005
- https://www.geeksforgeeks.org/python/dsa/